

GAME PROGRAMMING

War Thunder — Air Battle

Multiplayer Code Architecture Design

Student	Junaid Maqsood
SAP ID	59528
Course	Game Programming

1. Game Overview

War Thunder is a free-to-play massively multiplayer online military simulation game developed and published by Gaijin Entertainment. First released in 2013, it features combined arms combat across air, ground, and naval theatres of war, spanning vehicles from the 1930s through to the modern era. This document focuses exclusively on the Air Battle mode, which is the most mechanically distinctive and technically demanding of the three.

Core Air Battle Mechanics

In Air Battles, up to 32 players (16 per team) pilot historically accurate aircraft across three gameplay modes:

- **Arcade Battles:** simplified physics, auto-aim assist, shared respawn pool. Designed for accessibility.
- **Realistic Battles:** full realistic flight models (RFM), limited HUD, one life per aircraft. Demands skill.
- **Simulator Battles:** full cockpit-only view, no markers, external joystick recommended. Closest to real aviation.

Each air battle session consists of the following phases:

- Session lobby & matchmaking
- Mission briefing & aircraft selection
- Live aerial combat with real-time physics simulation
- Post-battle results, Research Points (RP) and Silver Lions (SL) distribution

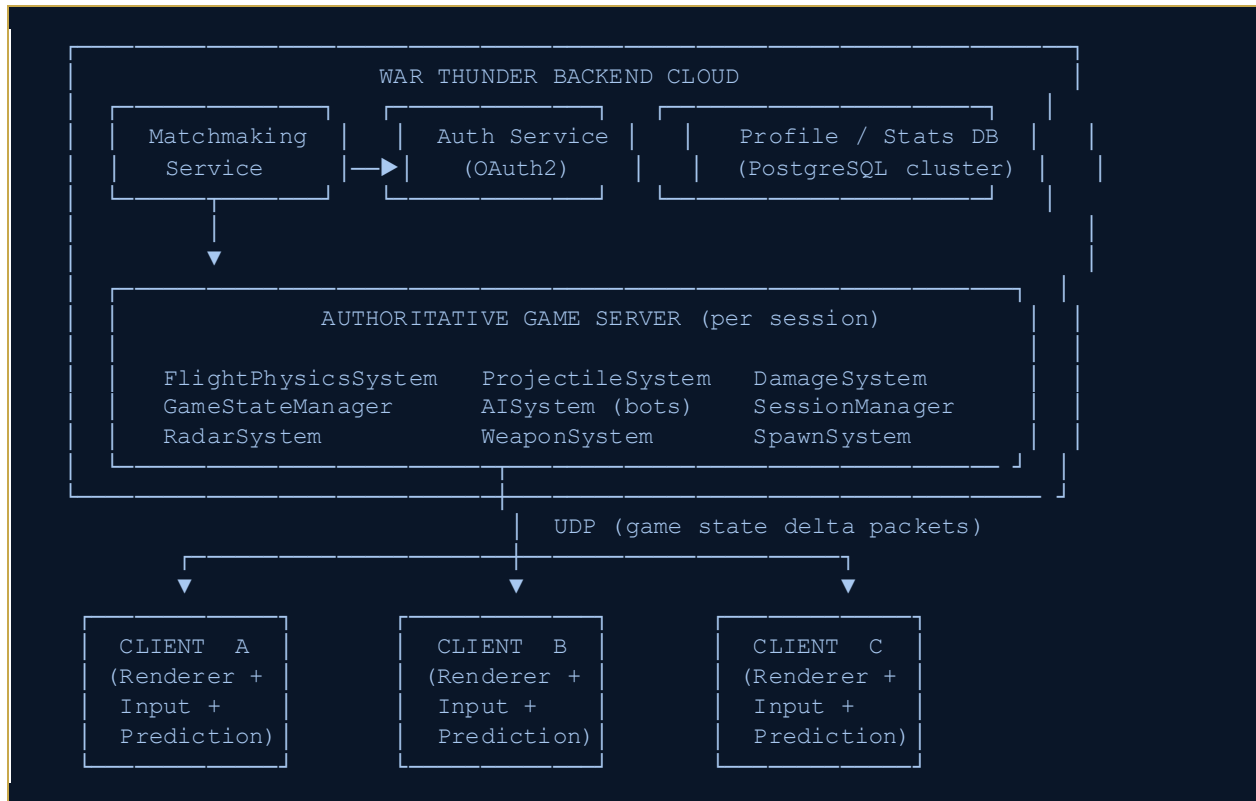
Key Technical Facts:

- Up to 32 concurrent players per air battle session
- Tick rate: ~30 Hz server simulation (Realistic/Simulator modes)
- Projectile physics simulated server-side (shell velocity, drag, penetration)
- Aircraft damage model: 9-zone structural damage (wings, fuselage, engine, tail, etc.)
- Radar & missile guidance computed per-frame on authoritative server

2. High-Level Architecture

War Thunder's air battle system is built on a Client-Server architecture with an Authoritative Game Server model. The server holds the single source of truth for all game state — positions, damage, ammunition, scores — and clients are treated as dumb input terminals that render what the server tells them, with client-side prediction layered on top to hide latency.

Architecture Overview Diagram



Major Systems

System	Responsibility
Player System	Manages pilot identity, aircraft loadout selection, team assignment, spawn state, and death/respawn lifecycle.
Matchmaking System	Balances teams by Battle Rating (BR), queues players, selects map/mission, spins up a dedicated game server instance.

Flight Physics System	Simulates the Realistic Flight Model (RFM) per aircraft: lift, drag, thrust, energy management, stall, compressibility.
Weapon System	Handles guns, bombs, rockets, and air-to-air missiles. Manages ammunition counts, reload times, weapon modes.
Projectile System	Simulates each fired shell or missile independently: trajectory, velocity decay, guidance logic (for missiles), hit detection.
Damage System	Evaluates penetration values vs armour, applies structural damage to aircraft zones, triggers fire/fuel leak/engine loss.
Radar System	Simulates radar sweep, lock-on, RWR alerts, chaff/flare countermeasures for jet-era aircraft.
Game State Manager	Maintains master snapshot of all entities each tick, computes delta packets, broadcasts to clients.
Session Manager	Controls session lifecycle: lobby → active → results. Handles disconnects, reconnects, and early-leave penalties.
UI System	Client-side only. Renders HUD (speed, altitude, heading, ammo, radar), kill feed, minimap, and post-battle screen.

3. Code Structure

The codebase is organized around three logical tiers: the Backend Services layer, the Authoritative Game Server, and the Game Client. Each tier has a clean separation of concerns to allow independent scaling and testing.

3.1 Codebase Directory Structure

```
war-thunder-air/  
├── backend/  
│   ├── auth/                # OAuth2, JWT session tokens  
│   ├── matchmaking/         # BR-based queue, lobby, server spin-up  
│   ├── profile/              # Pilot stats, hangar, research tree  
│   └── economy/              # SL / RP rewards, premium calculation  
├── game-server/              # Authoritative simulation (C++ / Dagor Engine)  
│   ├── core/  
│   │   ├── GameLoop.cpp      # Fixed 30 Hz tick loop  
│   │   ├── EntityManager.cpp # ECS entity registry  
│   │   └── EventBus.cpp       # Decoupled inter-system messaging  
│   ├── systems/  
│   │   ├── FlightPhysics/    # RFM, aerodynamics per aircraft  
│   │   ├── WeaponSystem/     # Gun bursts, bomb release, missile fire  
│   │   ├── ProjectileSystem/  # Per-projectile trajectory & hit detection  
│   │   ├── DamageSystem/     # Zone damage, fire, structural failure  
│   │   ├── RadarSystem/      # Sweep, lock, RWR, countermeasures  
│   │   ├── SpawnSystem/      # Spawn points, respawn timer, BR validation  
│   │   └── SessionManager/    # Lifecycle, scoring, disconnect handling  
│   ├── network/  
│   │   ├── NetServer.cpp      # UDP socket management  
│   │   ├── StateDelta.cpp     # Snapshot diff & compression  
│   │   └── LagCompensation.cpp # Rewind & re-simulate for hit validation  
│   └── ai/  
│       └── BotController.cpp   # Bot AI for incomplete lobbies  
└── client/  
    ├── rendering/            # 3D scene, cockpit, effects (Dagor Engine)  
    ├── input/                 # Keyboard / mouse / joystick mapping  
    ├── prediction/            # Client-side prediction + reconciliation  
    ├── audio/                 # Engine sounds, weapons, environment  
    └── ui/                    # HUD, menus, hangar, battle results
```

3.2 Entity-Component System (ECS)

The game server uses an Entity-Component System to represent all in-game objects uniformly. Every aircraft, projectile, bomb, and marker is an Entity composed of data Components. Systems operate over components in batch, keeping logic and data separated for performance and testability.

Aircraft Entity — Components

Component	Data Held
TransformComponent	World position (x, y, z), rotation quaternion, velocity vector
AircraftStatsComponent	Max speed, climb rate, turn rate, BR rating, fuel capacity
FlightStateComponent	Current throttle, flap setting, gear state, energy (speed × altitude)
HealthComponent	HP per zone: left wing, right wing, fuselage, engine, tail, pilot
WeaponLoadoutComponent	Gun ammo counters, bomb pylons, missile hardpoints, reload timers
RadarComponent	Radar mode (search/lock), current locked target, RWR threat list
NetworkComponent	Owner client ID, last acknowledged tick, input buffer
TeamComponent	Team ID (Allies/Axis), spawn slot index

Projectile Entity Components

Component	Data Held
TransformComponent	Position, velocity vector, age in ticks
ProjectileStatsComponent	Calibre, muzzle velocity, drag coefficient, explosive mass
GuidanceComponent	Missile only: seeker type (IR/SARH/ARH), current target entity ID, G-limit
OwnerComponent	Firing entity ID, team ID (for friendly fire checks)

3.3 Key Classes & Responsibilities

GameLoop (game-server/GameLoop.cpp)

Drives the entire server simulation at a fixed 30 Hz tick rate using a high-resolution timer. Each tick it:

- Reads all pending client input packets from the network buffer
- Advances FlightPhysicsSystem for every aircraft entity

- Steps ProjectileSystem — moves each projectile, checks collisions
- Runs DamageSystem on confirmed hits
- Updates RadarSystem sweep states
- Evaluates win conditions via SessionManager
- Snapshots world state and dispatches delta packets to all clients

FlightPhysicsSystem

Implements War Thunder's Realistic Flight Model (RFM). Each aircraft has a unique aerodynamic profile defined in a data file. Each tick the system:

- Reads throttle, pitch, roll, yaw inputs from the aircraft's input buffer
- Calculates lift force (using angle-of-attack and airspeed), drag, and thrust
- Checks for stall condition (AoA > critical angle) and applies buffeting
- Integrates velocity and position using a semi-implicit Euler integrator
- Clamps altitude at terrain collision and applies G-force damage to pilot

ProjectileSystem

Handles ballistics for all fired weapons. For guns, a simple drag model is applied each tick. For air-to-air missiles, a guidance pipeline runs:

- Pure Pursuit (early IR missiles): seeker points directly at heat signature
- Proportional Navigation (modern AAMs): adjusts heading based on line-of-sight rotation rate
- Active Radar Homing (ARH): autonomous after lock-on, resistant to flares

Hit detection uses a swept-sphere test against aircraft bounding volumes, then passes confirmed hits to DamageSystem.

DamageSystem

Resolves hits against the target aircraft's zone health pool. For cannon shells it calculates penetration using the formula:

$$\text{Penetration} = (\text{shell_mass} \times \text{velocity}^2) / (2 \times \text{armour_hardness} \times \text{calibre}^2)$$
 If $\text{penetration} \geq \text{zone_armour_value} \rightarrow \text{zone takes full damage + spall damage to adjacent zones}$
 Fire chance calculated from $\text{fuel_proximity} \times \text{shell_incendiary_rating}$

StateDelta (network/StateDelta.cpp)

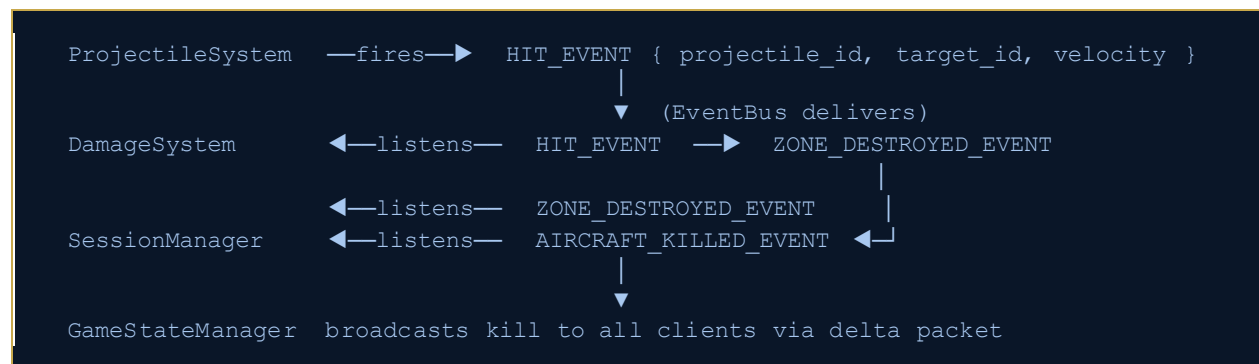
Rather than broadcasting the full world state every tick (which would be ~100 KB per tick for 32 players), StateDelta computes a binary diff between the previous acknowledged snapshot and the current snapshot per client. Only changed component fields are transmitted, compressed with LZ4. Typical packet size: 1–4 KB per tick per client.

LagCompensation (network/LagCompensation.cpp)

When a client reports a hit, the server rewinds the world state by the client's measured round-trip time, re-simulates the shot from the client's recorded position, and validates whether the hit was geometrically possible. This prevents fast-moving targets from being unhittable due to network delay, while also blocking impossible hit claims from cheaters.

3.4 Inter-System Communication

Systems do not call each other directly. They communicate through a central EventBus (publish-subscribe pattern) to maintain decoupling:



Event	Description
HIT_EVENT	Fired by ProjectileSystem when swept-sphere collision is confirmed. Carries projectile stats for damage resolution.
ZONE_DESTROYED_EVENT	Fired by DamageSystem when a structural zone reaches 0 HP. Triggers visual effects on client.
AIRCRAFT_KILLED_EVENT	Fired by DamageSystem when all critical zones are destroyed. Notifies SessionManager for score update.
INPUT_RECEIVED_EVENT	Fired by NetServer when a client packet arrives. Queued to aircraft input buffer for next tick.
SESSION_END_EVENT	Fired by SessionManager when win condition is met. Triggers reward calculation and client notification.

4. Design Decisions

4.1 Why Authoritative Server Model?

War Thunder is a competitive game where millimetre-accurate hit detection and fair projectile physics are fundamental to the experience. A peer-to-peer architecture would allow any client to manipulate its own physics simulation, enabling cheating (speed hacks, aimbot trajectory spoofing). The authoritative server model ensures that only the server's simulation result is canonical. The client is only trusted for control inputs; all outcomes are computed server-side.

4.2 Why ECS Over Traditional OOP?

A traditional object-oriented design would model an Aircraft as a monolithic class containing its own physics, damage, and weapon logic. With 32 aircraft plus hundreds of active projectiles per session, this creates scattered memory access patterns that destroy CPU cache efficiency. The Entity-Component System stores each component type in a contiguous array, so the FlightPhysicsSystem iterates only over flight-relevant data without touching weapon or radar data it does not need. This data-oriented design yields significantly better performance at scale.

4.3 Why UDP Over TCP?

Air combat is latency-sensitive. A 100ms delay in registering a joystick input or receiving an aircraft position update is clearly perceptible and disrupts gameplay. TCP's retransmission-and-order-guarantee mechanism introduces head-of-line blocking: if one packet is lost, all subsequent packets wait. For real-time game state, a slightly stale position update is far preferable to waiting for retransmission. War Thunder uses UDP with a custom reliability layer that only guarantees delivery for critical events (kills, session end), while position updates are simply re-sent each tick — if one is lost, the next tick's update supersedes it naturally.

4.4 Why Battle Rating for Matchmaking?

Rather than a traditional Elo-based skill matchmaking, War Thunder uses Battle Rating — a value assigned to each aircraft based on its real-world performance envelope (speed, armament, maneuverability). This is architecturally simpler and more predictable: the matchmaking service only needs to compare numeric BR values and enforce a maximum spread (typically ± 1.0 BR). This also respects the simulation-first philosophy of the game: a P-51D Mustang should never face an F-86 Sabre, regardless of how skilled the pilot behind the Mustang is.

4.5 Why EventBus for Inter-System Communication?

Directly coupling systems creates a dependency web that makes testing and iteration difficult. If DamageSystem directly called SessionManager.recordKill(), both systems would need to exist

in every test context. The EventBus decouples producers from consumers: DamageSystem fires AIRCRAFT_KILLED_EVENT without knowing or caring who is listening. New systems (e.g., an AchievementSystem or an AnalyticsSystem) can subscribe to existing events without modifying any existing code. This follows the Open/Closed Principle and dramatically simplifies adding new game features.

4.6 Why Client-Side Prediction?

Even with UDP, a player in Lahore connecting to a European server faces ~100-150ms round-trip latency. Without prediction, every joystick input would feel like flying through mud — you move the stick and 150ms later the aircraft responds. Client-side prediction runs a local copy of the flight physics immediately on input, so the aircraft responds instantly. When the server's authoritative tick arrives, the client reconciles any divergence by smoothly interpolating (rubberband correction). This makes the game feel responsive on any reasonable internet connection, which is critical for the game's global player base.